

Building the Test Generator

How the framework was assembled across four repositories — which files had to change, and what was actually broken in each case. Reconstructed from commit history, not memory.

Four repositories

Three are forks of existing projects. Only the driver layer was written from scratch — the strategy throughout was to make existing tools work against the RISC-V model rather than to replace them.

10x-Engineers/isla The symbolic execution library fork of rem-s-project/isla	10x-Engineers/isla-testgen The test generator built on it fork of the CHERI/Morello generator
10x-Engineers/sail-riscv-testgen The Golden Model fork of riscv/sail-riscv	10x-Engineers/riscv-test-generation Drivers, sweeps, measurement written from scratch
10x-Engineers/sail-riscv-autotest The concrete oracle backend written from scratch	

The sequence

Two independent tracks ran in parallel and met in the driver layer: a concrete oracle that needed nothing from the symbolic engine, and a symbolic path that needed changes in three separate repositories before it produced a single test.

5 JANUARY 2026 · ISLA

Groundwork in the symbolic library

Before any RISC-V target existed, three changes made the library usable at all for a new architecture: an `--isla-output-dir` option so `isla-sail` could write its IR somewhere chosen, an extension of the list-nil handling to `eq_anything`, and constant primops for a 32-bit target.

`f742610` `bf098be` `b898a55`

2 JULY 2026 · AUTOTEST

The concrete oracle, in one commit

24 files. The oracle track needed nothing from the symbolic engine: run a random instruction sequence on the Golden Model, read the resulting architectural state back, and bake it into a self-checking test. That independence is why floating point and vector had a working generator long before the symbolic side could touch them.

FILE	RESPONSIBILITY
<code>config.py</code>	read a Golden Model configuration
<code>model.py</code>	derive the instruction set from the Sail sources
<code>oracle.py</code>	run the model, read the state back
<code>selfcheck.py</code>	turn observed state into assertions
<code>program.py</code>	render assembly
<code>toolchain.py</code>	assemble and link exactly as the model does
<code>suite.py</code> / <code>runner.py</code>	organise by extension; run and summarise

6f3f4a1

16 JULY 2026 · ISLA

Making the library compile against the real model

Seven files, four unrelated reasons. This is the commit that took the symbolic engine from "does not build" to "runs the RISC-V model".

FILE	WHY IT HAD TO CHANGE
<code>isla-sail/jib_ir.ml</code> <code>isla-sail/sail_plugin_isla.ml</code>	Upstream Sail API drift. <code>Jib.funtype</code> 's <code>bool</code> tag became a variant, <code>CT_bit</code> was removed, and the <code>rewrite-pass</code> list had gained, lost and reordered entries. <code>isla-sail</code> is a Sail <i>plugin</i> , so it must track whatever the Sail it links against looks like.
<code>isla-lib/src/primop.rs</code>	Primops the model calls that the library never implemented: <code>cancel_reservation</code> , <code>valid_reservation</code> , <code>match_reservation</code> , <code>load_reservation</code> , <code>count_trailing_zeros</code> , <code>vector_init</code> , <code>sys_enable_experimental_extensions</code> .
<code>isla-lib/src/bitvector.rs</code> <code>bitvector/b64.rs</code> , <code>b129.rs</code>	<code>BV::trailing_zeros</code> , required by the new primop.
<code>isla-lib/src/memory.rs</code>	<code>check_concrete_overlap</code> hardcoded 64-bit addresses. RV32 addresses are 32 bits, so reads were silently mis-sized. Now uses the address's own solver-tracked width.

The one that mattered most. `vector_access` and `vector_update` had no case for struct-typed elements, because `smt_value` cannot represent `Val::Struct`. RISC-V's PMP configuration is `vector(64, Pmpcfg_ent)` — a vector of bitfield structs, not bitvectors — so any symbolic access to a PMP register crashed the engine. The fix builds a per-field ITE

instead. Without it there is no symbolic PMP testing at all, and PMP is a required item in the RFP.

00ffeb4

16 JULY 2026 · SAIL-RISCV

The model side: entry points and a codegen workaround

The generator needs a two-phase init/step calling convention, which the model did not expose. `isla_testgen_init()` and `isla_testgen_step()` were added to `main.sail`, modelled on the existing `isla_footprint` entry points, and `fetch()` was made non-private so the latter could call it.

Then a compiler bug got in the way. Any Sail function binding `_` or a destructuring pattern to a parameter of two or more tuple elements got two inconsistent synthetic names out of `isla-sail`. Worked around by destructuring in the function body instead of the parameter list — about 50 functions across `core/`, `sys/`, `extensions/` and `pmp/`.

Four functions hit the same bug in the scattered-function *merging* step rather than in any clause body, so `read_CSR`, `write_CSR`, `is_CSR_accessible` and `is_CSR_exception_virtual` — 302 function clause s across ~17 files — were fully de-scattered into one plain function each, preserving first-match-wins order.

`reset_pmp()` also had to build a fully concrete `Mk_Pmpcfg_ent(zeros())` per entry rather than a `with`-update on the prior symbolic value. Together with the `vector_access` fix above, this is what unblocked PMP.

7a6fb873

16 JULY 2026 · ISLA-TESTGEN

The RISC-V target itself

The generator is organised around a `Target` trait of some 28 methods — register naming, PC alignment, entry points, ELF emission. Adding RISC-V meant implementing it, writing an object emitter, and fixing three assumptions in the shared execution engine that were true only for the architectures already supported.

FILE	WHAT IT DOES
<code>src/target.rs</code>	the <code>RiscV</code> Target implementation
<code>src/generate_object_riscv.rs</code>	ELF emitter, 405 lines
<code>src/execution.rs</code>	three generic fixes — see below
<code>riscv-ir/riscv32.ir</code>	the compiled model IR, 451,420 lines

Three assumptions that were not portable.

Concrete boot PC. The engine set the initial PC by asserting equality at the solver level. That silently does nothing when the PC is already concrete — which it is, because RISC-V's init

calls `reset()` . The frame kept the wrong PC. Now the value is overwritten directly, which works whether it was symbolic or concrete.

PC width vs address width. RV32's PC is 32 bits, but the model's `sail_mem_read` interface uses 64-bit addresses regardless of XLEN. The PC is now zero-extended to `addr_size()` for the fetch.

Register numbering. GPRs were assumed to start at zero. RISC-V's `x0` is hardwired to zero and is not a general-purpose register, so a `first_gpr_index()` was added to the trait.

4f7b031

21-22 JULY 2026 · PARENT

The driver repository opens

The parent repo is created, the submodule is pointed at the RISC-V branch, and the pipeline is documented as a flow diagram. At this point both engines exist but nothing drives them systematically.

3432eb7 86fe2bd 001a200

11 AUGUST 2026 · PARENT

The driver and measurement layer

One commit brings the whole Python framework under version control — eight scripts and the documentation trail behind them. This is where generation stops being a manual invocation and becomes a sweep with a coverage figure attached.

SCRIPT	WHAT IT ANSWERS
<code>model_opcodes.py</code>	which instructions exist — parsed from the model, not from riscv-opcodes
<code>opcode_sweep.py</code>	generate and run one per instruction
<code>csr_sweep.py</code>	the ~10 privileged extensions that define no mnemonics at all
<code>scenario_tests.py</code>	architectural state, not single instructions
<code>coverage_report.py</code>	how much of the model the corpus reaches
<code>coverage_matrix.py</code>	per-instruction status across both generators
<code>status_report.py</code>	whose fault each failure is
<code>regression.py</code>	what got worse since the baseline

9ab92bf

11 AUGUST 2026

Privileged capability, and making the repo clonable

Per-ELF coverage, PMA scenarios driven by the model's own config, and interrupt delivery. Separately, `paths.py` replaces absolute paths that had been repeated across four scripts, and the model becomes a submodule.

Two things the model taught us here. `--pending-interrupt` deliberately left `mstatus.MIE` clear, so it tested WFI rather than delivery. Adding `--deliver-interrupt` exposed that delegation had to use a *supervisor* software interrupt: `legalize_mideleg` hardwires the machine-mode bits to zero, so delegating a machine software interrupt writes 0 and the trap is still taken in M-mode. The first version did exactly that and read as a delegation failure when it was an impossible test.

59b0fde b1d8230 eade459 4238a61 549dd5c

24 AUGUST 2026

Floating point, downstream compatibility, and a clone that verifies itself

The symbolic engine gains a SoftFloat interface, closing the last capability gap; both backends start emitting the riscv-arch-test contract so the two halves produce interchangeable output; and the repository gains a single command that proves a fresh clone works.

CHANGE	WHY
primop/softfloat.rs	every FP operation in the model calls SoftFloat; isla had no binding, so FP terminated with <code>NoFunction</code>
smtlib.rs / smt.rs	<code>FPUnary::ToIEEE</code> , the inverse of the existing <code>FromIEEE</code> , wired to Z3
generate_object_riscv.rs	scratch registers could collide with the harness's own, clobbering the value under test — a green test that verified nothing
program.py / selfcheck.py	ACT header, signature region and <code>rvtest_entry_point</code> , so one file runs under both runtimes
tools/verify_clone.py	runs the pipeline end to end rather than checking that files exist

29cf051 9b6498f 0a812d5 6c2ef42

What it added up to

REPOSITORIES	FORKED	WRITTEN NEW	MONTHS
5	3	2	8

Three of the five repositories are forks, and every change in them was made to reach the RISC-V model rather than to alter what those tools do. The changes divide cleanly into four kinds: **API drift** in the Sail plugin, **missing primops** the model calls, **portability assumptions** that were true only for 64-bit zero-based-GPR targets, and **one genuine engine gap** — struct-typed vector elements — without which PMP could not be tested symbolically at all.

The model fork is the one worth being precise about. It carries a single commit, and nothing in it changes architectural behaviour: two entry points, a workaround for an `isla-sail` codegen bug affecting roughly 50 functions, four scattered functions consolidated,

and one initialisation made concrete. But they *are* changes, which is why the technical plan does not describe the model as unmodified.

Reconstructed from the commit history of all five repositories. Every commit referenced is reachable on the 10x-Engineers forks.